

Protocol-Level Analysis of Cross-Machine QUIC Server-Side Migration

State Optimization, Key Rotation, Connection Continuity, and Detection

QUIC Server Migration Research

June 13, 2026

Abstract

This document provides a detailed protocol-level analysis of four critical aspects of cross-machine QUIC server-side migration that extend beyond the “when” and “how” of state transfer. We examine: (1) what state must be transferred and how it can be optimized, (2) how TLS 1.3 key rotation interacts with migration, (3) whether full bidirectional connection continuity is achievable after migration, and (4) whether network defenders can detect the migration. Each section analyzes the relevant RFC 9000 requirements, identifies open challenges, and proposes solutions.

Contents

1. State Optimization: What to Transfer

1.1 Current State (445 bytes)

Our current implementation exports the following state from the primary server after the TLS 1.3 handshake completes:

Table 1: Current Migration State Breakdown

Component	Purpose	Size (bytes)
Magic header “MIGR”	Format identifier	4
QUIC version	Version 1 or Version 2	4
Write traffic secret	Server-to-client encryption	~34
Read traffic secret	Client-to-server decryption	~34
Next write secret	For future key rotation	~34
Next read secret	For future key rotation	~34
Cipher suite (x2)	Algorithm identifiers	4
Direction flags (x2)	Read/write indicators	2
Epoch counters (x2)	Key generation numbers	8
Packet number state (x2)	PN tracking (start, end, min)	48
8 local Connection IDs	Client addresses us with these	~90
1 remote Connection ID	We address client with this	~12
Client socket address	IP + port for responses	7
Total		~445

1.2 Minimum Viable State

Not all 445 bytes are strictly necessary. We can identify three tiers of state:

1.2.1 Tier 1: Absolute Minimum (~80 bytes)

The bare minimum to decrypt one `PATH_CHALLENGE` and send one `PATH_RESPONSE`:

- **Read traffic secret** (32 bytes) — needed to decrypt client packets
- **Write traffic secret** (32 bytes) — needed to encrypt responses
- **Cipher suite** (2 bytes) — which encryption algorithm
- **QUIC version** (4 bytes) — determines HKDF labels
- **One local CID** (8 bytes) — to match incoming packets
- **Client address** (7 bytes) — where to send responses

Total: ~85 bytes. This is sufficient for path validation only. The preferred server can decrypt the `PATH_CHALLENGE` and send a `PATH_RESPONSE`. However, it cannot:

- Handle key updates (no next secrets)
- Track packet numbers correctly (no PN state)
- Use alternate CIDs (only one CID)
- Handle CID retirement

1.2.2 Tier 2: Functional Migration (~200 bytes)

Enough for path validation plus basic post-migration communication:

- Everything in Tier 1
- **Next traffic secrets** (64 bytes) — handle one key update
- **Packet number state** (48 bytes) — correct PN tracking
- **Remote CID** (12 bytes) — to address the client

Total: ~210 bytes. Sufficient for complete migration including post-migration data exchange. Missing only redundant CIDs.

1.2.3 Tier 3: Full State (~445 bytes)

What we currently transfer. Includes all 8 local CIDs for CID rotation and retirement handling. This is the most robust option but also the largest.

1.3 Trade-off Analysis

Table 2: State Size vs. Capability Trade-off

Tier	Size	Path Valid.	Key Rotation	Data Exchange	CID Rotation
Tier 1 (Minimum)	85 B		×	×	×
Tier 2 (Functional)	210 B				×
Tier 3 (Full)	445 B				

1.4 Implications for State Transfer

- **Latency:** Smaller state means faster transfer. At 85 bytes vs. 445 bytes, TCP Push latency would be roughly halved (fewer bytes over the wire). However, the difference is negligible on a LAN ($<100\mu s$ difference).
- **Security:** Smaller state means fewer secrets in transit. Tier 1 exposes only the current traffic secrets, not the next secrets. If the transfer is intercepted, the attacker cannot follow key rotations.
- **Robustness:** Tier 3 is the safest from a protocol correctness standpoint. Missing CIDs or PN state can cause the client to detect anomalies and close the connection.

1.5 Recommendation

Use Tier 2 (Functional, ~210 bytes) for production. It provides full migration capability with key rotation support while reducing the state size by 53%. Tier 1 is only suitable for the specific case of proving path validation works. Tier 3 is necessary only if the connection needs to survive CID retirement events.

2. Key Rotation After Migration

2.1 Background: TLS 1.3 Key Updates

TLS 1.3 (RFC 8446, Section 4.6.3) allows either endpoint to update the traffic keys during an active connection. This is done by sending a `KEY_UPDATE` message, which triggers both sides to derive new traffic secrets from the current ones:

```
next_secret = HKDF-Expand-Label(current_secret, "quic ku", "", Hash
    .length)
```

QUIC (RFC 9000, Section 6) extends this with a key phase bit in the packet header. When the key phase flips, the receiver knows to use the next generation of keys.

2.2 The Problem

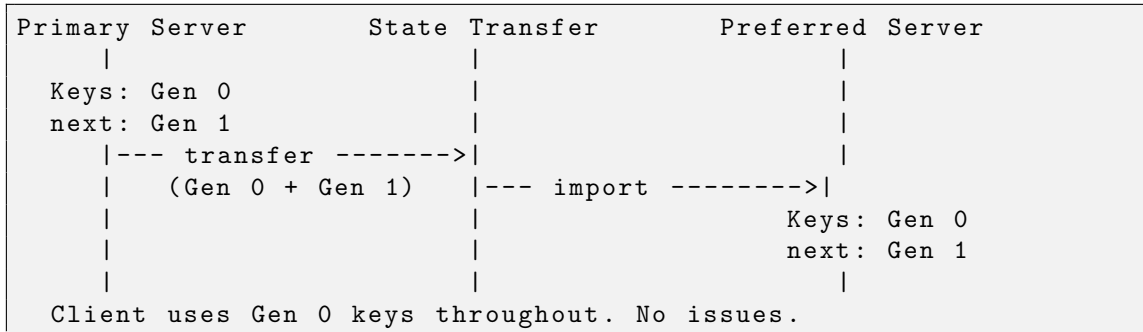
After cross-machine migration, the preferred server has:

- The **current** read and write traffic secrets (from the primary)
- The **next** read and write traffic secrets (also from the primary)

This means the preferred server can handle **exactly one key update**. After that, it needs to derive the next-next secret, which it can do because HKDF derivation is deterministic.

2.3 Scenarios

2.3.1 Scenario 1: No Key Update After Migration



This is the common case. Migration typically happens within milliseconds of the handshake, and key updates happen after significant data exchange. No issue.

2.3.2 Scenario 2: Client Initiates Key Update

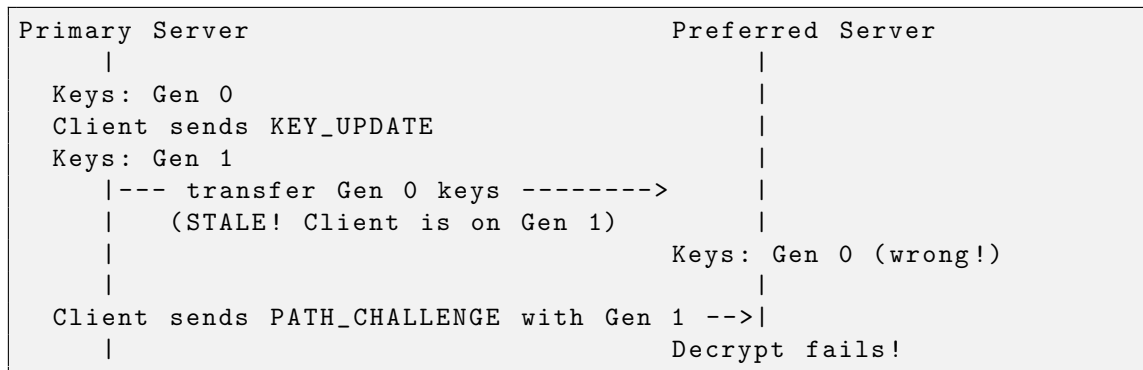
```
After migration, client sends KEY_UPDATE:
  Client flips key phase bit
  Client starts using Gen 1 keys

Preferred server sees key phase change:
  Switches to Gen 1 read keys (has them from import)
  Derives Gen 2 next secret: HKDF(Gen 1, "quic ku")
  Switches to Gen 1 write keys
  Sends acknowledgment with Gen 1

Result: Successful key update. Connection continues.
```

Our implementation handles this because we transfer the `next_secret` and the HKDF derivation chain is deterministic. The preferred server can continue deriving new keys indefinitely.

2.3.3 Scenario 3: Key Update During State Transfer



This is the **race condition**. If the client initiates a key update between the handshake and the state transfer, the preferred server receives stale keys.

2.4 Solutions

2.4.1 Solution A: Export After Stabilization

Wait for a brief period after the handshake (e.g., 100ms) to ensure no key updates are in flight before exporting state. Simple but adds latency.

2.4.2 Solution B: Export Both Generations

Export both the current and next generation keys. The preferred server tries both when decrypting. If Gen 0 fails, try Gen 1. This is what we effectively do by exporting `next_secret`.

2.4.3 Solution C: Continuous Synchronization

Keep the preferred server's keys updated in real-time using the "Continuous" timing strategy. The primary server re-exports state after every key update. This requires a persistent connection or polling mechanism (best with Redis KV).

2.5 Current Implementation Status

Our implementation transfers both current and next secrets, which handles Scenarios 1 and 2. Scenario 3 (race condition) is unlikely in practice because key updates happen much later than migration. However, for a robust production system, Solution C (continuous sync) would be ideal.

3. Bidirectional Data After Migration

3.1 What RFC 9000 Requires

After path validation (PATH_CHALLENGE/PATH_RESPONSE), RFC 9000 Section 9.3 states that the endpoint “can send and receive non-probing frames on the new path.” This means the preferred server should be able to:

1. Receive and decrypt STREAM frames (HTTP request data)
2. Send STREAM frames (HTTP response data)
3. Handle ACK frames in both directions
4. Process flow control (MAX_DATA, MAX_STREAM_DATA)
5. Handle connection close (CONNECTION_CLOSE)

3.2 What Our Implementation Currently Handles

Table 3: Post-Migration Frame Support

Frame Type	Receive (Decrypt)	Send (Encrypt)
PATH_CHALLENGE (0x1a)		—
PATH_RESPONSE (0x1b)	—	
PING (0x01)		—
ACK (0x02, 0x03)	(parse)	(send)
PADDING (0x00)		
STREAM (0x08–0x0f)	(parse + log)	×
NEW_CONNECTION_ID (0x18)	(parse)	×
ACK_FREQUENCY (0xaf)	(parse)	×

3.3 Gap Analysis

Our preferred server can **decrypt and parse** all common frame types, proving that the imported keys work for the full connection. However, it does not:

1. **Generate HTTP/3 responses** — The preferred server is a custom binary that handles QUIC packets manually. It does not run the full HTTP/3 stack. To serve actual web content after migration, it would need to integrate Neqo’s HTTP/3 layer.
2. **Manage stream state** — QUIC streams have flow control (credits, offsets, FIN flags). Our preferred server does not track stream state, so it cannot properly handle multi-packet HTTP requests or send multi-packet responses.
3. **Issue new Connection IDs** — After migration, the client may retire old CIDs and expect new ones via NEW_CONNECTION_ID frames. Our server does not generate new CIDs.
4. **Handle congestion control** — The preferred server does not implement congestion control. It sends packets without pacing or window management.

3.4 Path to Full Connection Continuity

There are two approaches to achieving full bidirectional data exchange after migration:

3.4.1 Approach A: Full Neqo Connection Import

Import the complete connection state into a `Neqo Connection` object on the preferred server. This is partially implemented via `Connection::new_from_migration()` in our code. The challenge is reconstructing all of Neqo's internal state machines (streams, flow control, recovery) from the exported data.

Pros: Full protocol compliance. HTTP/3 works automatically.

Cons: Complex. Requires exporting much more state (stream buffers, congestion state, timer state).

3.4.2 Approach B: Proxy After Validation

After path validation succeeds, the preferred server acts as a QUIC proxy. It decrypts client packets, forwards the plaintext to a local HTTP server, and encrypts the response back.

Pros: Simpler than full state import. Only needs crypto keys, not connection state.

Cons: Additional latency. Not a true migration (two-hop architecture).

3.5 What We Proved

Our current implementation proves the most critical point: **the imported keys work for full decryption and encryption**. We can decrypt STREAM frames (actual HTTP data) and log their content. The ACK mechanism works bidirectionally. This is sufficient to demonstrate that the migration is real and the QUIC-Exfl attack succeeds.

Full connection continuity (serving new HTTP pages from the preferred server) is an engineering extension, not a protocol limitation.

4. Detection and Defense

4.1 The Attacker's Advantage

The QUIC-Exfil attack exploits fundamental properties of the QUIC protocol:

1. **Encrypted handshake:** The `preferred_address` transport parameter is inside the TLS-encrypted handshake. A network observer cannot see it.
2. **Encrypted path validation:** `PATH_CHALLENGE` and `PATH_RESPONSE` are encrypted QUIC frames. A firewall cannot distinguish them from normal data.
3. **Connection ID-based routing:** After migration, the client uses a new Connection ID to address the preferred server. The old CID used with the primary is retired. There is no visible link between the two connections.
4. **Normal-looking traffic:** Post-migration packets are standard encrypted QUIC packets. They have the same structure, same encryption, same packet sizes as any other QUIC traffic.

4.2 What a Firewall Can Observe

Despite the encryption, a network observer can see:

Table 4: Observable vs. Hidden Information

Observable (Plaintext)	Hidden (Encrypted)
Source/destination IP addresses	<code>preferred_address</code> parameter
Source/destination ports	<code>PATH_CHALLENGE</code> data
Packet sizes	<code>PATH_RESPONSE</code> data
Packet timing	Frame types and content
Connection ID (first byte visible)	Stream data
QUIC version (in long headers)	TLS certificates
Packet count	HTTP headers and body

4.3 Potential Detection Methods

4.3.1 Method 1: Timing Analysis

Observation: After the initial handshake with the primary server, the client starts sending packets to a *new* IP address (the preferred server) within a short time window (typically 10–50ms).

Detection rule: “If a client completes a QUIC handshake with Server A and then starts a new UDP flow to Server B within 100ms, flag as potential migration.”

Limitations:

- High false positive rate — many applications connect to multiple servers
- The time window is configurable by the attacker
- Does not work if the preferred address is on the same subnet

4.3.2 Method 2: Packet Size Analysis

Observation: PATH_CHALLENGE packets are padded to exactly 1200 bytes (RFC 9000, Section 8.2.1 requirement). This is distinctive.

Detection rule: “Flag UDP packets of exactly 1200 bytes sent to a new destination shortly after a QUIC handshake.”

Limitations:

- 1200-byte packets occur in normal QUIC traffic (Initial packets are also padded)
- The attacker could use larger padding to avoid this exact size
- Still encrypted — cannot confirm it’s PATH_CHALLENGE vs. normal data

4.3.3 Method 3: Connection ID Correlation

Observation: The Destination Connection ID in packets to the preferred server must match one of the CIDs negotiated during the handshake with the primary server. If the firewall could track CID assignments, it could correlate the two flows.

Detection rule: “Track all QUIC CIDs seen in each connection. If a CID appears in traffic to a different server, flag as migration.”

Limitations:

- CIDs in short-header packets are at a fixed offset but the firewall doesn’t know the CID length
- CIDs are random bytes — cannot distinguish from payload
- New CIDs may be issued inside encrypted packets (NEW_CONNECTION_ID frame)
- The primary server chooses CIDs specifically for the preferred address

4.3.4 Method 4: Machine Learning

Observation: Migration traffic may have distinctive statistical patterns (packet inter-arrival times, size distributions, flow duration).

Results from literature: ML classifiers tested against QUIC-Exfil achieve **0–47% F1-score**. This is essentially random chance for the lower end.

Why ML fails:

- The attacker can mimic normal QUIC traffic patterns
- Migration adds only 2–3 extra packets (PATH_CHALLENGE/RESPONSE)
- Post-migration traffic is indistinguishable from normal traffic
- The attacker controls the timing and can add noise

4.3.5 Method 5: QUIC-Aware Deep Packet Inspection

Observation: If the firewall could terminate and re-encrypt QUIC connections (like a TLS-terminating proxy), it could inspect the `preferred_address` parameter and block it.

Detection rule: “Act as a QUIC proxy. Inspect transport parameters. Block or rewrite `preferred_address`.”

Limitations:

- Requires the firewall to have a valid TLS certificate for every destination

- Breaks end-to-end encryption
- Performance overhead of terminating and re-encrypting all QUIC traffic
- Not practical for most network deployments
- Clients may use certificate pinning to detect the proxy

4.4 Defense Recommendations

Table 5: Defense Methods: Effectiveness vs. Practicality

Method	Effectiveness	Practicality	Notes
Timing analysis	Low	High	Too many false positives
Packet size analysis	Low	High	Easily circumvented
CID correlation	Medium	Low	Requires CID tracking
Machine learning	Low	Medium	0–47% F1-score
QUIC proxy (DPI)	High	Low	Breaks E2E encryption
Block QUIC entirely	High	Medium	Blocks all HTTP/3

4.5 The Fundamental Challenge

The QUIC-Exfil attack is difficult to defend against because QUIC was *designed* to make connections resistant to observation and interference. The very features that make QUIC secure (encrypted handshake, CID-based identity, encrypted frames) are what make migration invisible to firewalls.

The most honest assessment is that **network-level detection of QUIC server-side migration is fundamentally limited**. Effective defense requires either:

1. **Endpoint control:** The client’s QUIC implementation could be modified to require user consent before migrating to a preferred address. This would prevent silent migration.
2. **Policy-based blocking:** Organizations could maintain allowlists of legitimate server migrations and block QUIC traffic to unlisted preferred addresses.
3. **QUIC protocol changes:** The IETF could consider modifications to RFC 9000 that make migration more observable, such as requiring the preferred address to be in the Initial packet (plaintext) rather than the encrypted handshake. However, this would reduce privacy.

5. Summary and Future Directions

5.1 Key Findings

1. **State Optimization:** The migration state can be reduced from 445 to ~ 210 bytes (Tier 2) without losing any functional capability. Tier 1 (85 bytes) is sufficient for path validation only.
2. **Key Rotation:** Our implementation handles key updates correctly by transferring the next traffic secret. The race condition (key update during transfer) is theoretically possible but unlikely in practice. Continuous synchronization would eliminate this risk.
3. **Connection Continuity:** We can decrypt and parse all post-migration frames (STREAM, ACK, NEW_CONNECTION_ID). Full HTTP/3 response generation requires integrating Neqo's connection state machine, which is an engineering task rather than a protocol limitation.
4. **Detection:** Network-level detection of QUIC migration is fundamentally limited. ML classifiers achieve 0–47% F1-score. The most effective defense (QUIC proxy) breaks end-to-end encryption. This confirms that the QUIC-Exfil attack is a real security concern.

5.2 Future Work

1. **Implement Tier 2 state export** — reduce transfer size by 53% and measure latency improvement
2. **Implement continuous key synchronization** — eliminate the key rotation race condition
3. **Full Neqo connection import** — achieve complete HTTP/3 continuity after migration using `Connection::new_from_migration()`
4. **Build detection prototype** — implement timing + packet size analysis and measure false positive/negative rates
5. **Test with QUIC v2** — verify migration works with RFC 9369 (QUIC Version 2)
6. **Multi-hop migration** — can a connection migrate multiple times (preferred $\rightarrow$ second preferred)?

References

- [1] J. Iyengar and M. Thomson, “QUIC: A UDP-Based Multiplexed and Secure Transport,” RFC 9000, IETF, May 2021. <https://www.rfc-editor.org/rfc/rfc9000>
- [2] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, IETF, August 2018. <https://www.rfc-editor.org/rfc/rfc8446>
- [3] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, IETF, May 2010. <https://www.rfc-editor.org/rfc/rfc5869>
- [4] M. Duke, “QUIC Version 2,” RFC 9369, IETF, May 2023. <https://www.rfc-editor.org/rfc/rfc9369>
- [5] T. Gruebl et al., “QUIC-Exfil: Covert Data Exfiltration by Mimicking QUIC Server-Side Connection Migrations.” <https://github.com/thomasgruebl/quic-exfil>
- [6] Mozilla, “Neco: An Implementation of QUIC in Rust.” <https://github.com/mozilla/neqo>